

How Model Context Protocol (MCP) Can Help Enterprises Scale and Manage AI Tools

Hari Patchigolla | Data Scientist @Autodesk



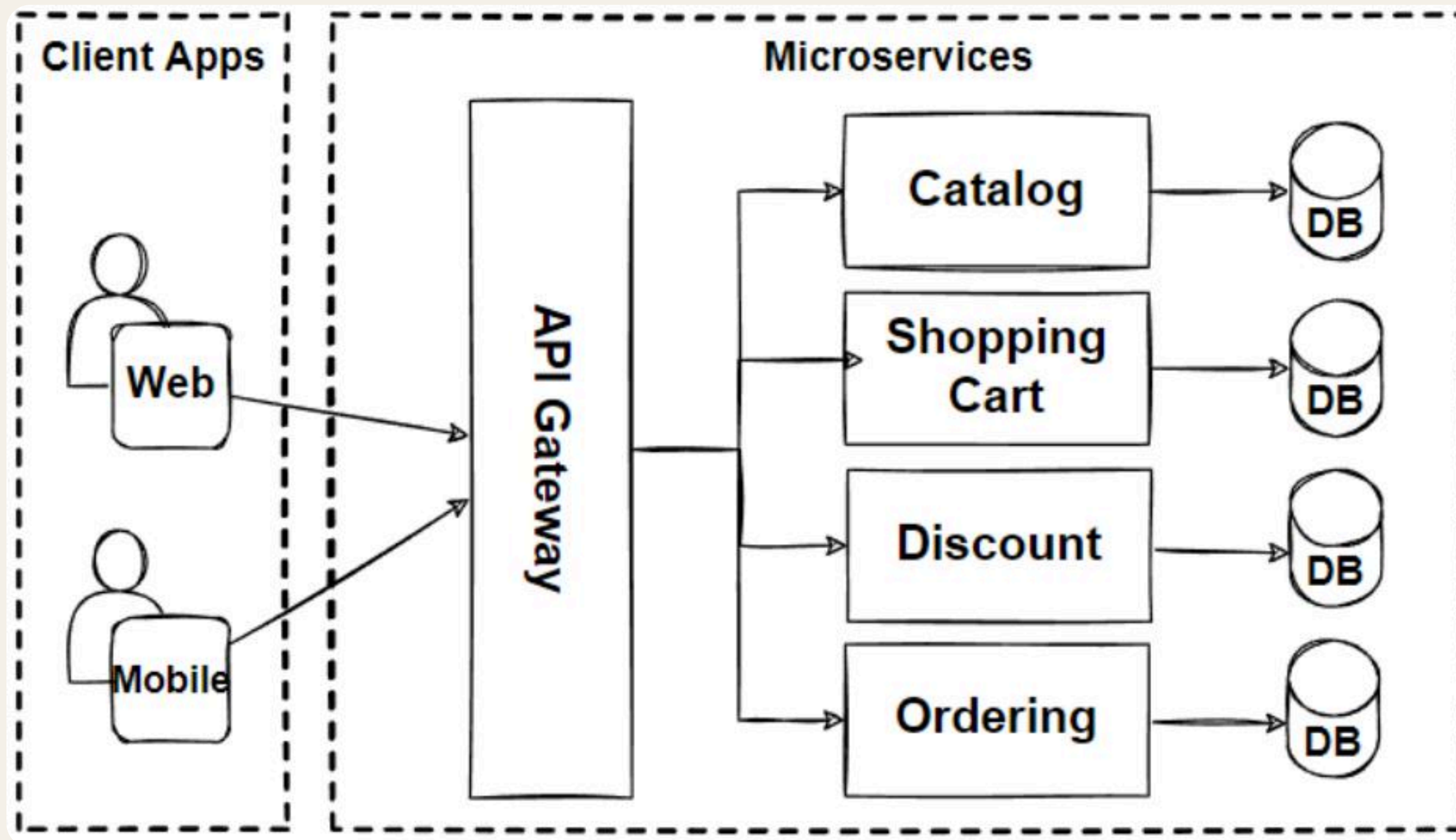
Today's Agenda:

1. Limitations of Current Agentic Systems
2. What is MCP? How does MCP address these issues?
3. Lets get coding.....!
 - a. mcp[cli]
 - b. FastMCP
 - c. PydanticAI

Limitations of Current Agentic Systems

01

Microservice Architecture in Software Development



● Modularity

Modular approach to building software as a suite of independently deployable services

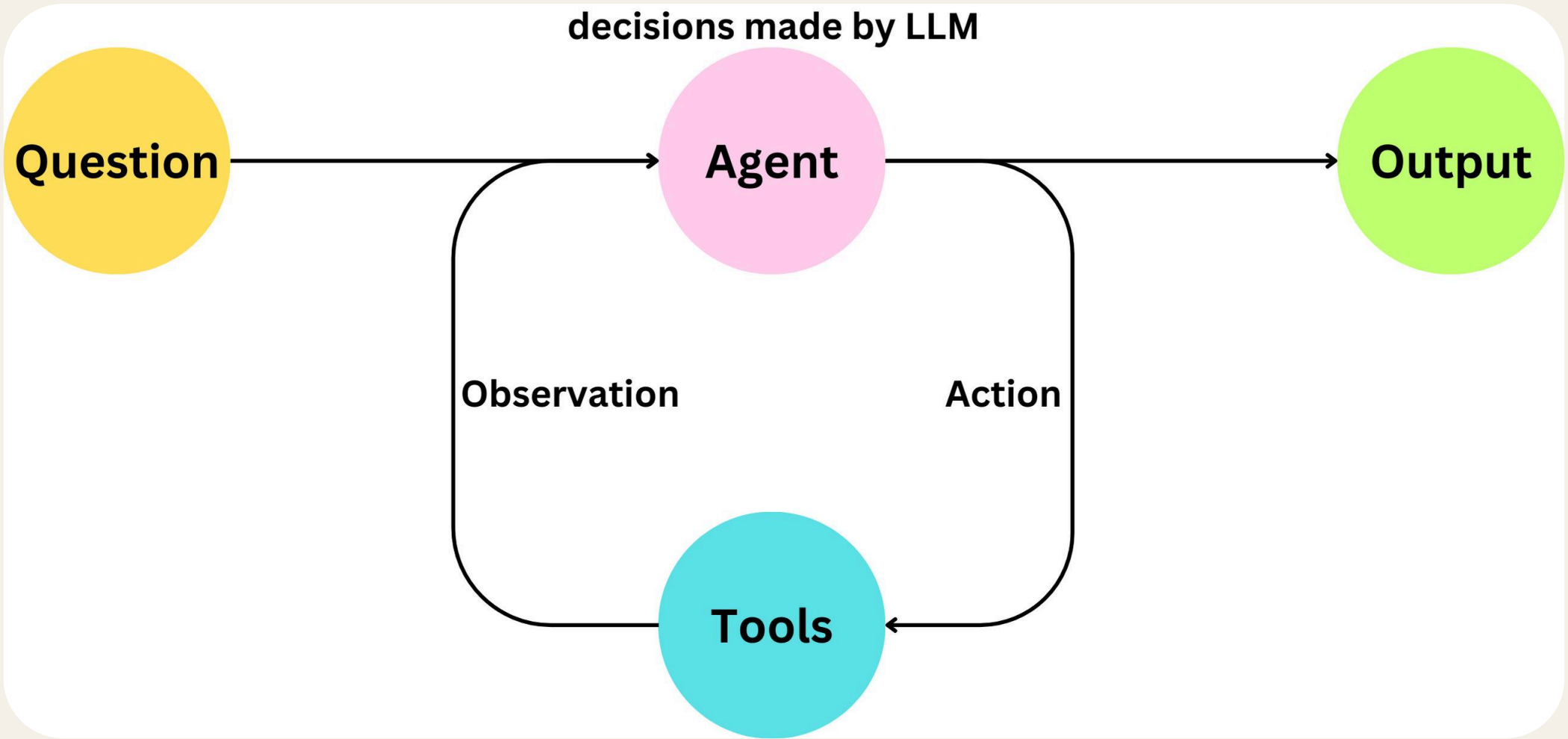
● Scalability

Enables scalability, flexibility, and independent development

● Maintainable

Common in modern enterprise systems for robust and maintainable solutions

Challenges in Current LLM Systems and Tool Integration



Tightly Coupled

LLMs often tightly coupled with tools; integration is not modular

Dependency

Changing a tool’s code may require restarting or redeploying the entire system

More Maintenance

Limited flexibility and increases maintenance overhead

Exponential System Growth

Exponential scaling of connections between AI models (M) and tools/data sources (N)

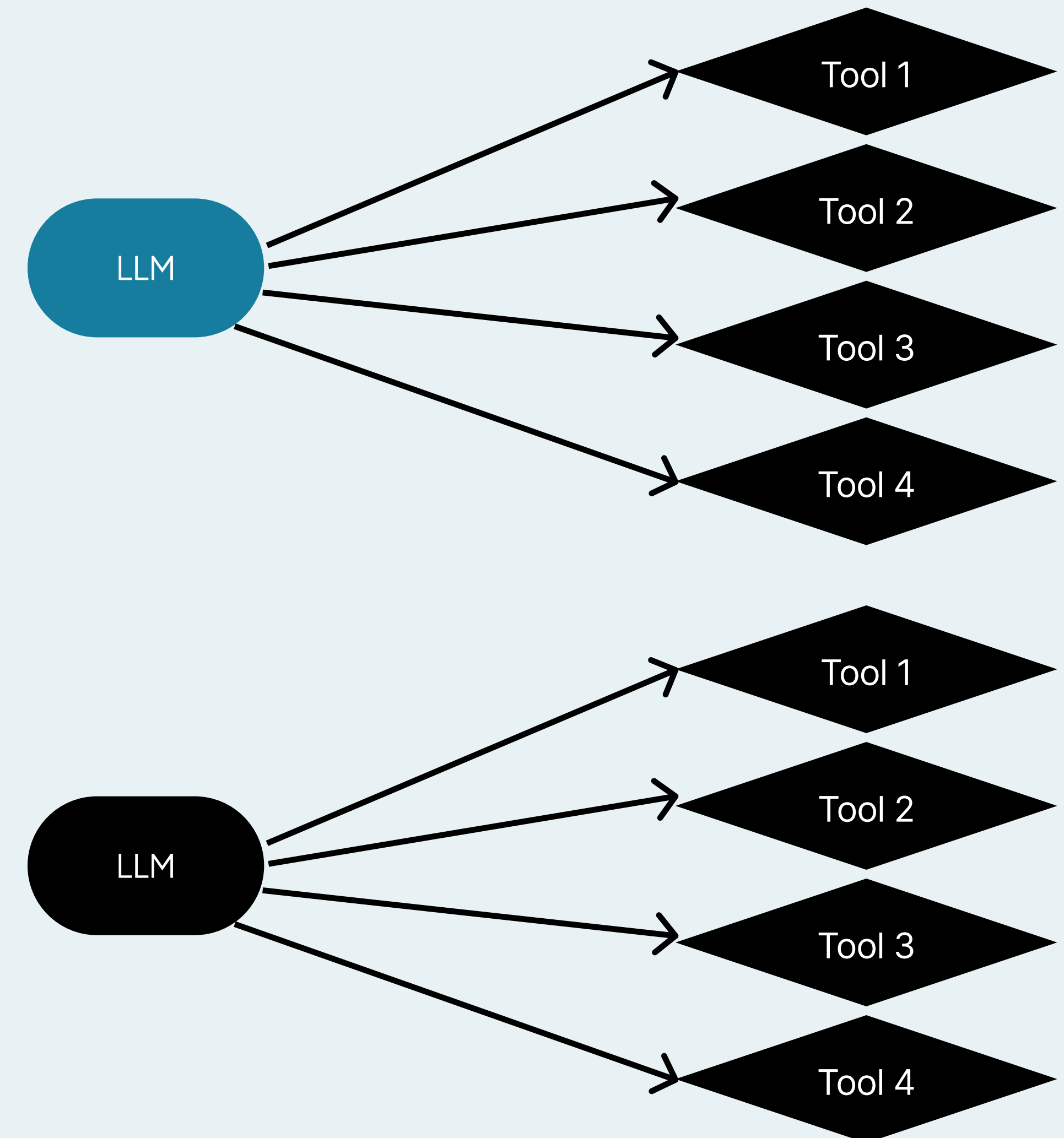
Tool Duplication

Traditional approach requires MxN custom integrations (5 models × 10 tools = 50 integrations)

Not Modular

Results in duplicated effort, inconsistent implementations, and maintenance nightmares

The MxN Problem



How this impacts team workflows



Heavy Team Dependency

An update to a tool makes every team have to update their code



Tool by Tool Logic

Each tool requires unique integration logic and authentication



System Restarts

Tools become inseparable from LLM workflows, requiring system-wide restarts or redeployments for updates



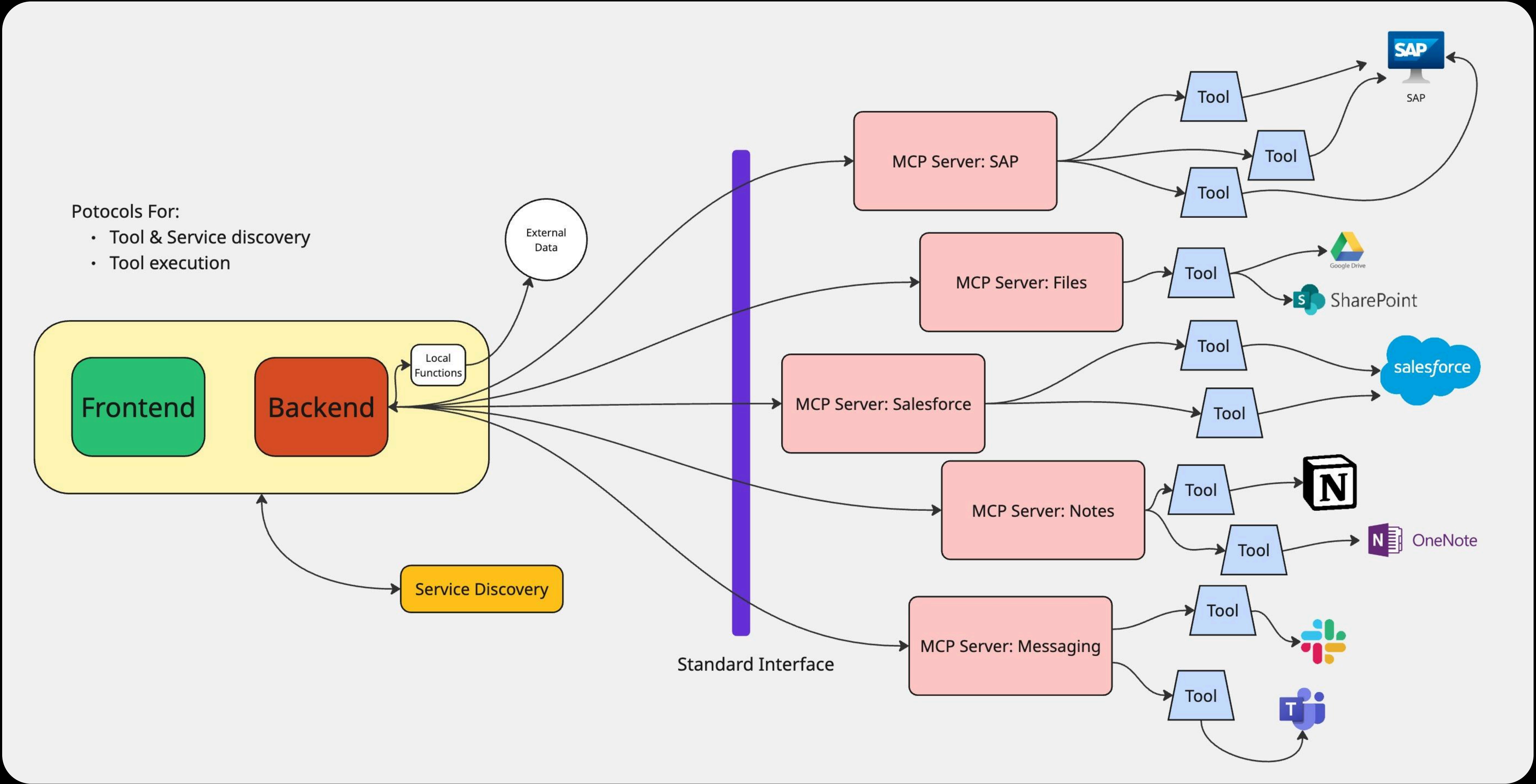
No Standardization

Difficulty in maintaining compliance and governance

O2

What is MCP? How
does MCP address
these issues?

1. An open protocol standardizing how LLMs, agents, and tools interact
2. Decouples tools from LLMs, enabling modular, plug-and-play integration



How MCP Address The Issues We Just Discussed...



Dynamic Tool Discovery

Agents find and use tools at runtime,
no hardcoding



Centralized tool governance

Manage access, policies, and
monitoring from one place



Linear Growth

Standardized protocol reduces
complexity to M+N implementations
(e.g. 5 models + 10 tools = 15 total
integrations)



Elastic Scalability

Easily add/remove agents and tools
without system disruption



Modularity

Reduces code duplication and
operational risk

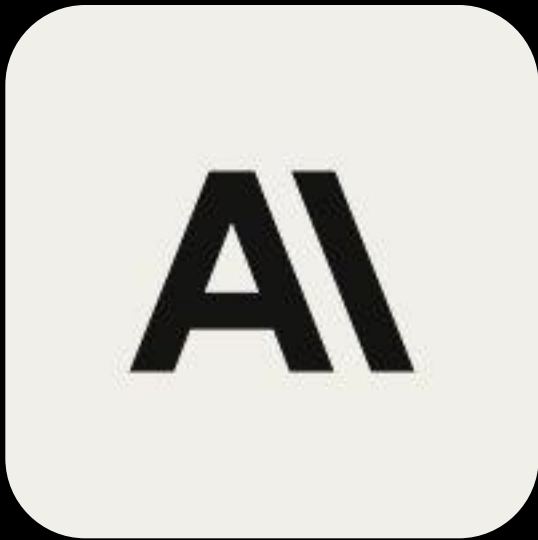
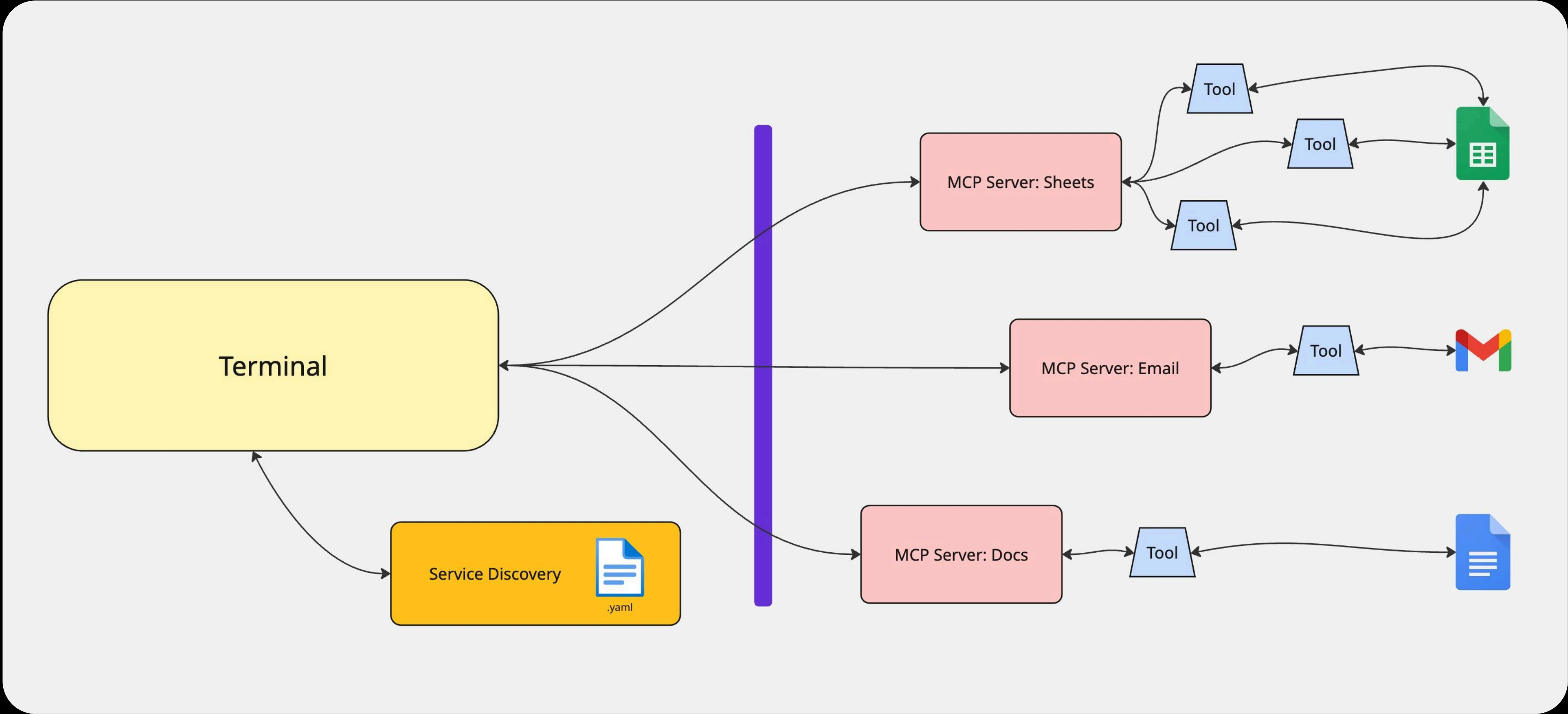


Compatibility

Enables any MCP client to work with
any MCP server

Lets Jump
Into The
Code!!!

03





Github
Repository

Let's Connect!



LinkedIn



Twitter/X

